

Architecture Microservices

Développement avec Spring Boot



Mouloud Menceur
mouloud.menceur@gmail.com

Dév. avec Spring Boot



Conteneur léger open-source pour construire les applications Java

Basé sur **l'inversion de contrôle et l'injection de dépendances**

Inversion de contrôle (IoC)

Le conteneur prend en charge l'exécution principale du programme
il coordonne et prend le contrôle sur l'application

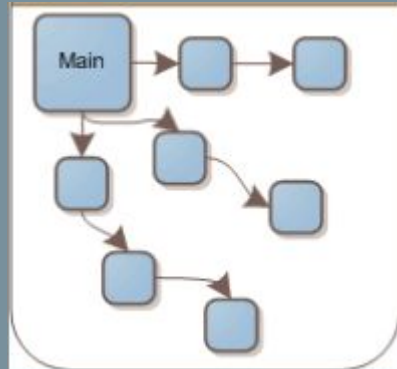
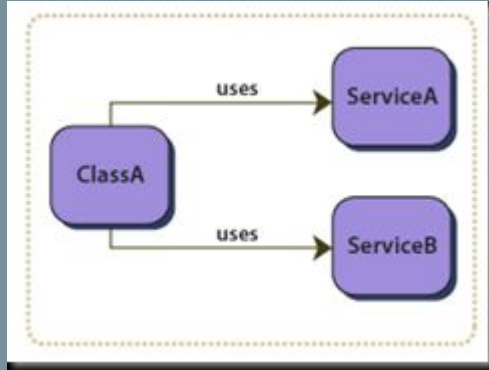
Injection de dépendances

Mécanisme implémentant l'IoC, créer dynamiquement les “dépendances” entre les
différents objets

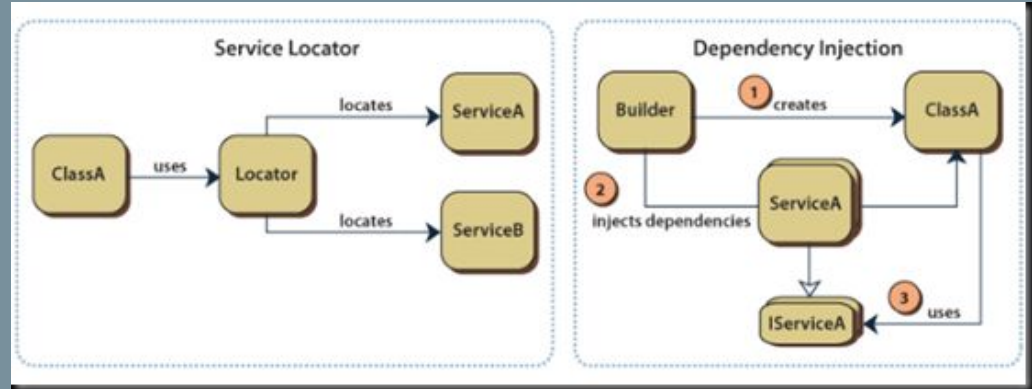
Principe **SOLID**

<https://spring.io/>

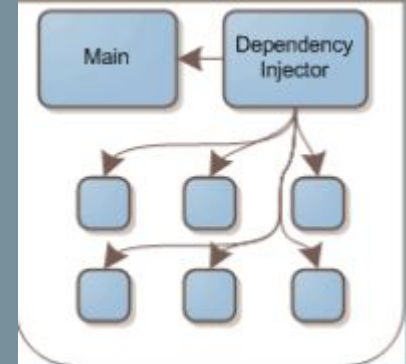
Dév. avec Spring Boot



Sans conteneur
IOC



conteneur IOC et Injection de
dépendances



<http://www.ytechie.com/2008/06/i-finally-get-the-point-of-inversion-of-control/>

Dév. avec Spring Boot

Solution modulaire (2004) : réponse à la complexité, lourdeur des serveurs Java EE...

Organisé autour de 20 modules

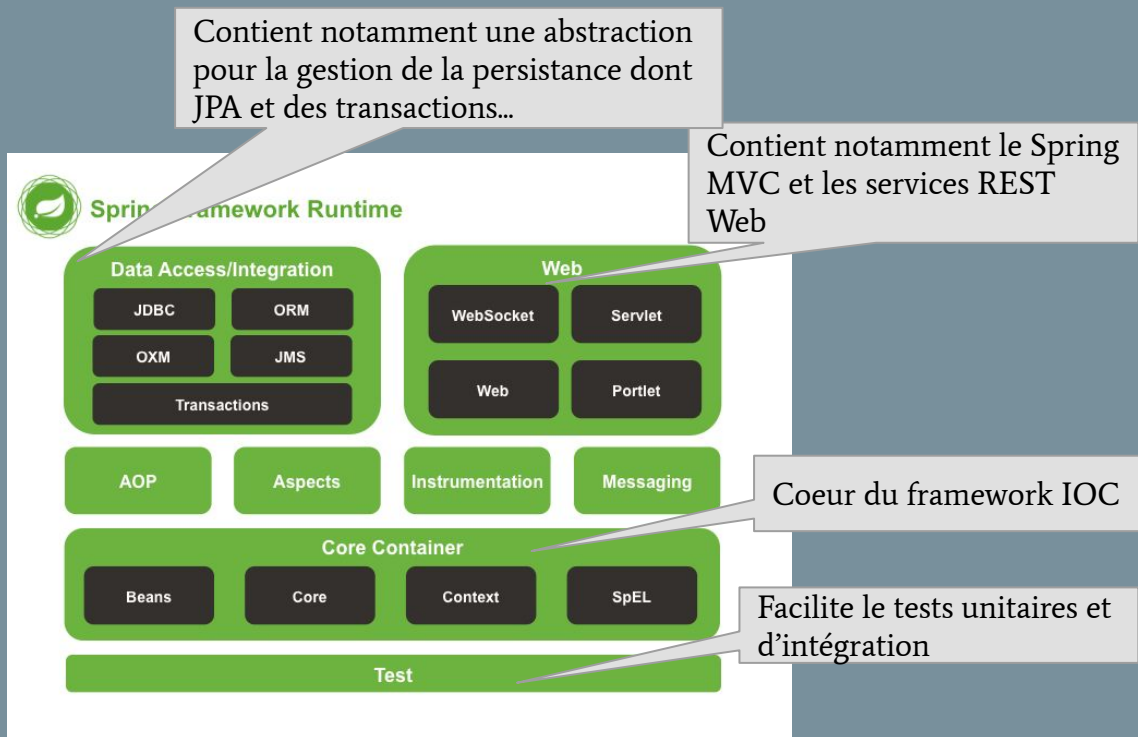
Dernière version Spring 7.0.7 (avril 2026)

... **Spring, framework riche et modulaire mais**

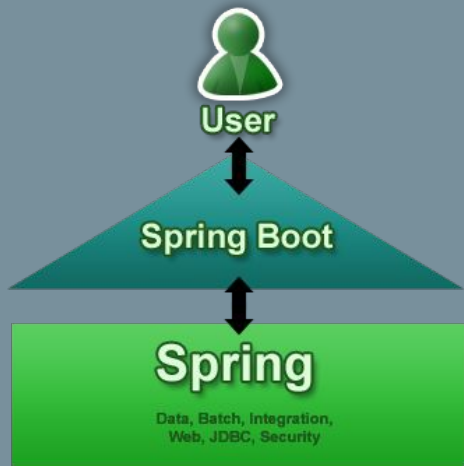
Configuration fastidieuse et complexe

Difficile de construire une application “production-ready”

La solution : Spring Boot



Dév. avec Spring Boot



<https://spring.io/blog/2013/08/06/spring-boot-simplifying-spring-for-everyone>

“Spring Boot aims to make it easy to create Spring-powered, production-grade applications and services with minimum fuss. It takes an **opinionated view of the Spring platform** so that new and existing users can quickly get to the bits they need. **You can use it to create stand-alone Java applications that can be started using ‘java -jar’** or more traditional WAR deployments. We also provide a command line tool that runs ‘spring scripts’.”

Dév. avec Spring Boot

Spring Boot

« Solution » Spring créé afin de faciliter la configuration et réduire le temps pour lancer un nouveau projet Spring. Basé sur le principe de « convention over configuration »
Peut-être vu comme une application Spring préconfigurée selon les meilleures pratiques

Principales fonctionnalités

- **Création d'applications autonomes** : <https://start.spring.io/> pour générer rapidement la structure du projet (**Maven** ou **Gradle**) avec toutes les dépendances nécessaires
- **Plus déploiement de .war** : serveur web (Tomcat ou Jetty) embarqué dans l'application
- **Regroupement de dépendances** maven afin de faciliter leur gestion (+ 150 starters)
- Fonctionnalités intégrées de supervision et de pilotage en production (Actuator)

Dév. avec Spring Boot

Installation de l'environnement de développement

1. Java

Installer OpenJDK 26

<https://jdk.java.net/26/>

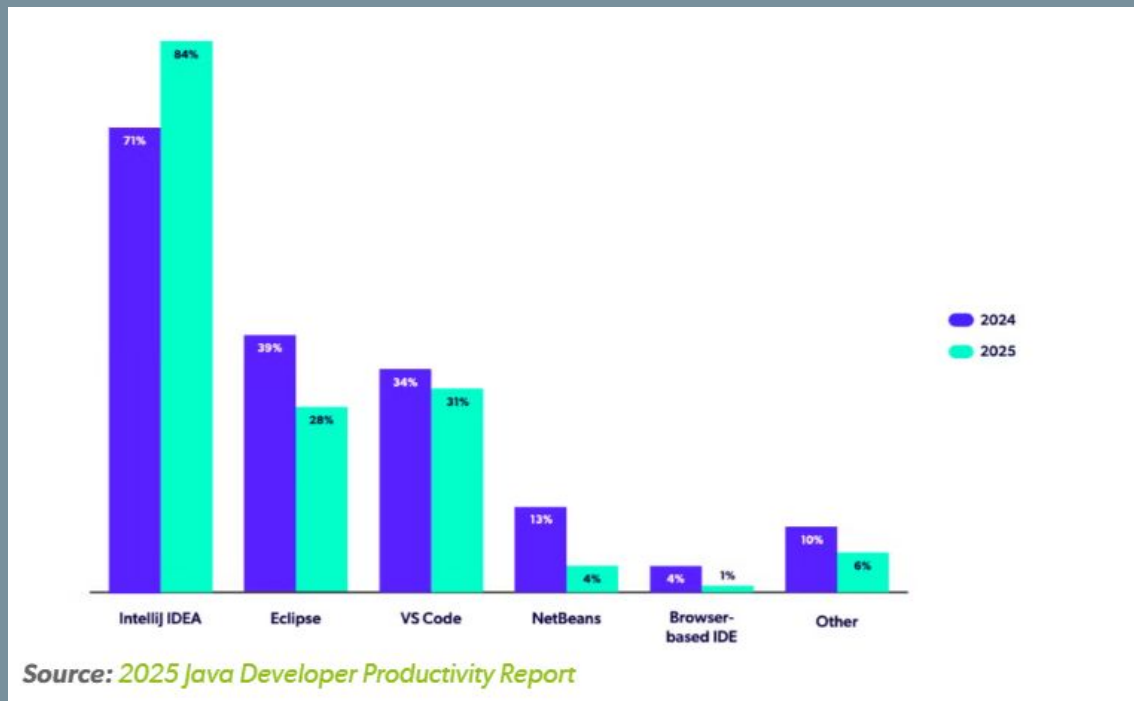
Attention : bien installer un JDK et pas uniquement un JRE

Créer la variable d'environnement JAVA_HOME (répertoire racine de l'installation du JDK)

2. Installation de IntelliJ IDEA

IDE Java <https://www.jetbrains.com/idea/download/>

Dév. avec Spring Boot



IDE Usage - <https://www.jrebel.com/blog/best-java-ide>

Dév. avec Spring Boot



Outil de la fondation Apache <https://maven.apache.org/> pour la construction (*build*) de projet Java

Objectif : produire un logiciel livrable à partir de ses sources en optimisant les tâches et en garantissant le bon ordre de fabrication :
compiler le code, gérer les dépendances vers les bibliothèques tierces et assembler pour produire le livrable mais aussi exécuter les tests, générer la documentation etc.

Maven repose sur l'approche "convention over configuration" :

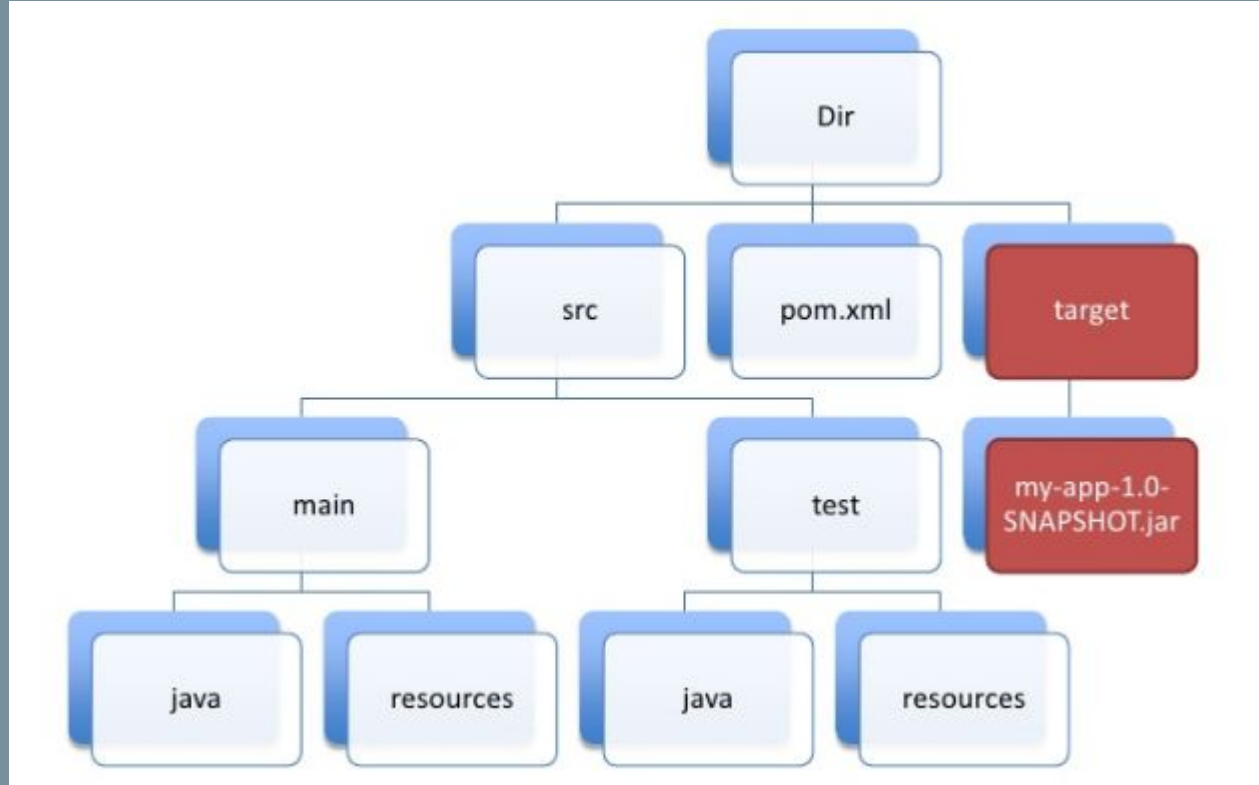
- **structure de projet standardisée** (arborescence de répertoires, nommage des fichiers...)
- **définition du cycle de vie d'un projet** (compiler, tester, packager...)

Gestion transitive des dépendances - dépendances accessibles depuis des repositories

Dév. avec Spring Boot

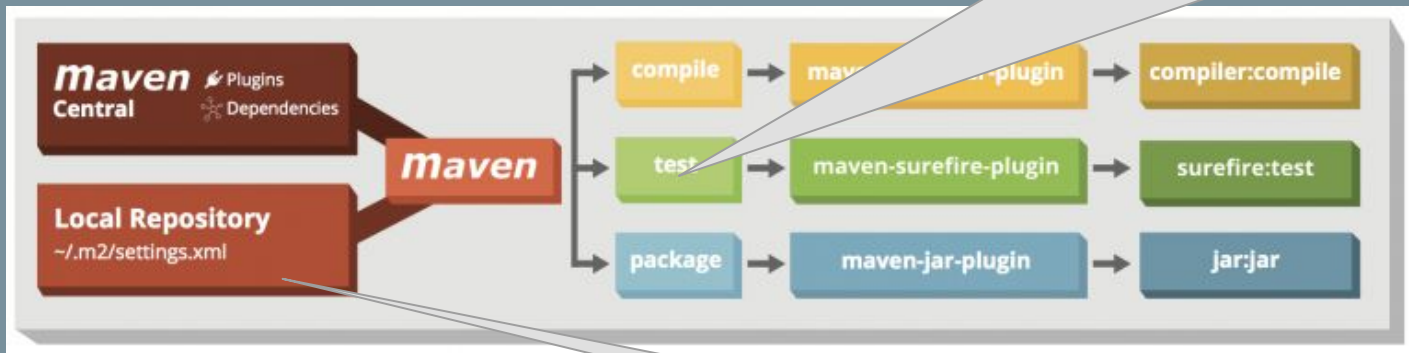
Structure standardisée des
répertoires d'un projet
Maven

<https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>



Dév. avec Spring Boot

Les “commandes” Maven séparent la construction du projet en différentes phases et forment le cycle de vie de construction du projet. Un système de plugins prend en charge l'exécution (d'une partie) du cycle de vie à travers des goals (actions)



<https://zeroturnaround.com/rebellabs/maven-cheat-sheet/>

Maven télécharge les “artefacts” (dépendances, plugins) depuis un dépôt distant et les cache dans le dépôt Maven local de la machine.

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>fr.dauphine.miageif.msa</groupId>
  <artifactId>MSA</artifactId>
  <version>1</version>
```

Projet Maven fr.dauphine.miageif.msa produit l'artefact exemple (un jar) en version 1

```
<dependencies>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.1.0</version>
  <scope>test</scope>
</dependency>
</dependencies>
</project>
```

Dépendance vers junit 5.1.0

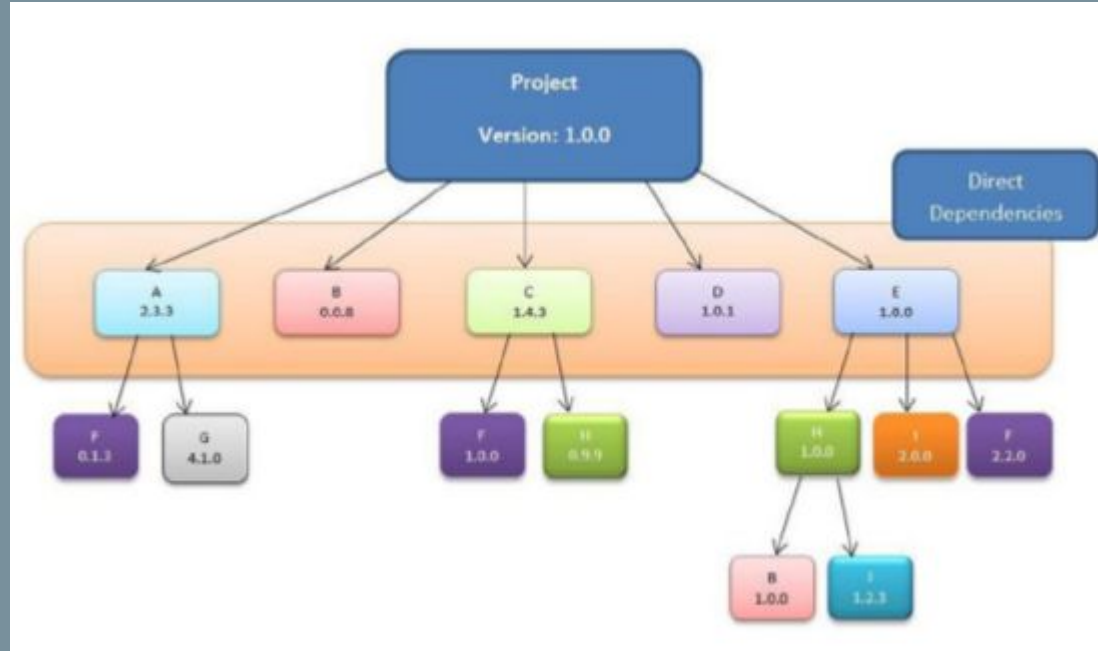
un fichier pom.xml à la racine du projet sert à décrire et configurer le projet Maven

- Chaque projet est configuré par un POM (Project Object Model) contenant les informations nécessaires (nom du projet, numéro de version, dépendances vers d'autres projets, bibliothèques nécessaires à la compilation...)
- **Supporte l'héritage des propriétés d'un projet parent**

Dév. avec Spring Boot

Gestion des dépendances transitives

Exemple : Le projet dépend directement de A, B, C, D et E. Comme E dépend H, I et F alors le projet en dépend également etc.



Source: [Introducing Maven. Apress](#)

Dév. avec Spring Boot

Installation de l'environnement de développement

3. Maven

Installer Maven (3.9.9) : téléchargement <https://maven.apache.org/download.cgi>

Installation par extraction de l'archive <https://maven.apache.org/install.html>

Ajouter le répertoire bin de l'installation Maven au PATH

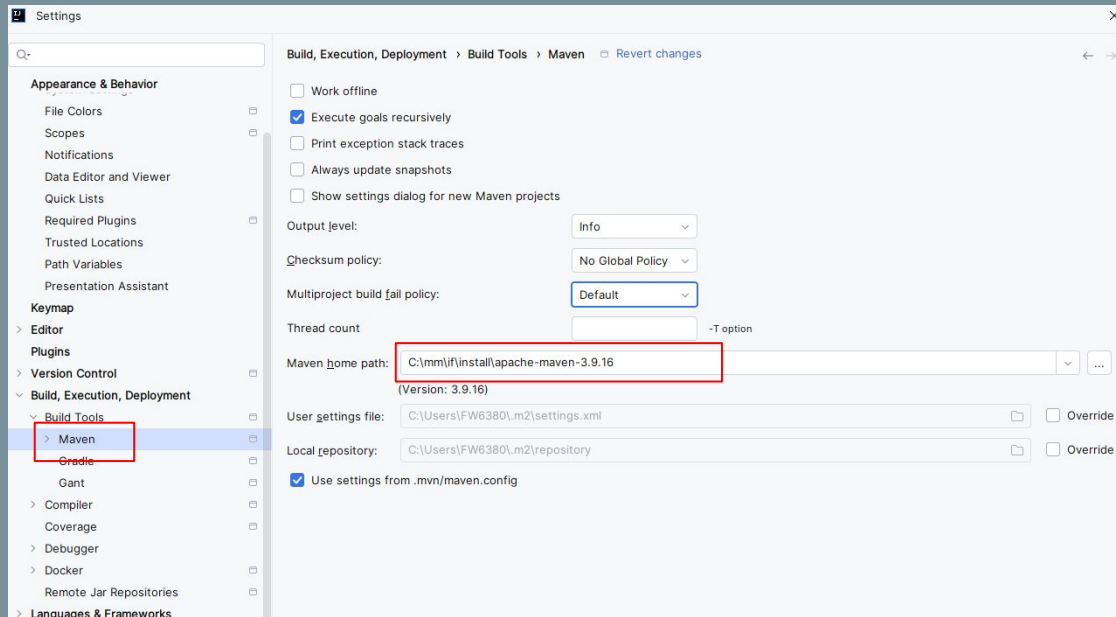
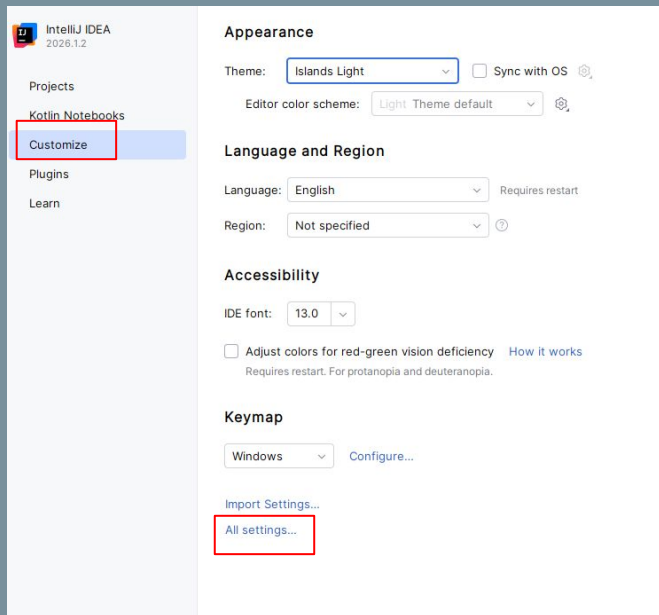
Lancer la commande `mvn -v`

```
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: C:\mm\if\install\apache-maven-3.9.16
Java version: 26.0.1, vendor: Oracle Corporation, runtime: C:\mm\if\install\openjdk-26.0.1\jdk-26.0.1
Default locale: fr_FR, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

Dév. avec Spring Boot

Installation de l'environnement de développement

3. Configuration Maven dans IntelliJ



Dév. avec Spring Boot

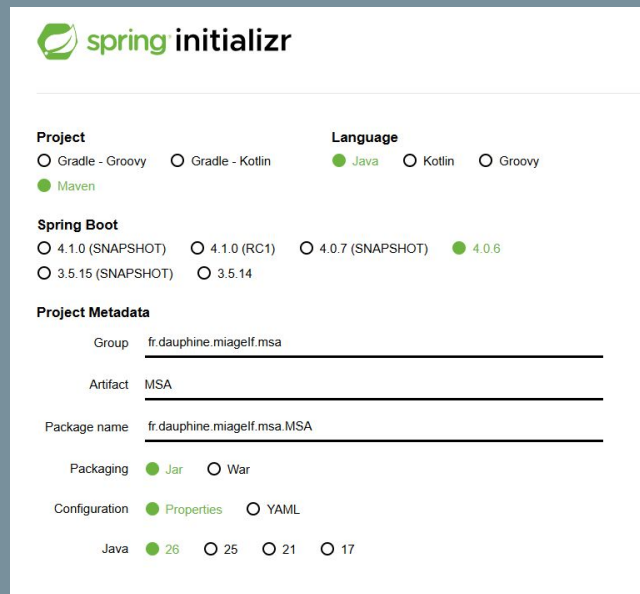
Initialisation du Projet avec Spring Initializr

Permet d'initialiser la construction du projet de l'application Spring en fonction des besoins
Directement depuis <https://start.spring.io/>

Projet Maven - Java - Spring Boot 4.0.6

Group : fr.dauphine.miageIf.msa

Artifact : MSA



The screenshot shows the Spring Initializr web form with the following configuration:

- Project:** ☒ Maven
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 4.1.0 (SNAPSHOT), ☐ 4.1.0 (RC1), ☐ 4.0.7 (SNAPSHOT), ☒ 4.0.6, ☐ 3.5.15 (SNAPSHOT), ☐ 3.5.14
- Project Metadata:**
 - Group: fr.dauphine.miageIf.msa
 - Artifact: MSA
 - Package name: fr.dauphine.miageIf.msa.MSA
- Packaging:** ☒ Jar, ☐ War
- Configuration:** ☒ Properties, ☐ YAML
- Java:** ☒ 26, ☐ 25, ☐ 21, ☐ 17

Dév. avec Spring Boot

On ajoute les dépendances suivantes :

Spring Web, DevTools, Spring Data JPA, H2, Actuator

“Generate” produit une archive MSA.zip

Archive à extraire dans un répertoire de développement

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

Spring Boot Actuator OPS
Supports built in (or custom) endpoints that let you monitor and manage your application - such as application health, metrics, sessions, etc.

Dév. avec Spring Boot

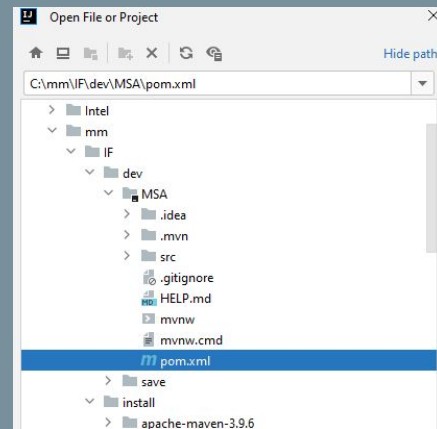
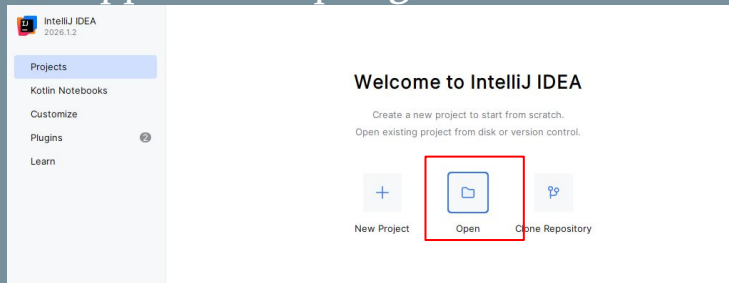
Initialisation du Projet avec Spring Initializr

Permet d'initialiser la construction du projet de l'application Spring en fonction des besoins
Directement depuis <https://start.spring.io/>

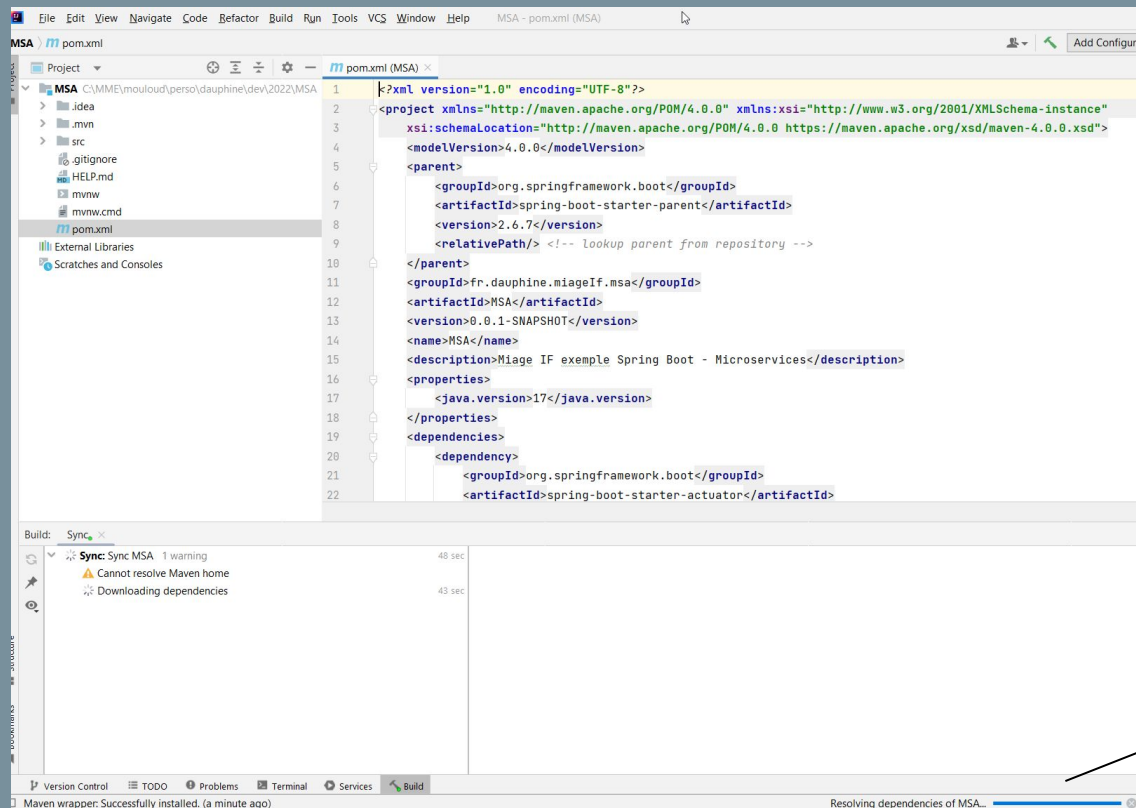
Import du projet Maven dans IntelliJ

Choisir **Open Project** et sélectionnez le répertoire de l'application exemple puis sélectionnez le fichier pom.xml

Analyse du fichier pom.xml



Dév. avec Spring Boot



Dépendances du projet

```
package fr.dauphine.miageif.msa.MSA;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
// Classe, générée automatiquement par Spring Boot, est le point d'entrée de
l'application
@SpringBootApplication
public class MsaApplication {

    public static void main(String[] args) {
        SpringApplication.run(MsaApplication.class, args);
    }
}
```

MsaApplication.java

Cette classe, générée automatiquement par Spring Initializr : point d'entrée de l'application
L'annotation **@SpringBootApplication** déclenche la configuration automatique de
l'infrastructure Spring

```
spring.application.name=devise-change  
server.port=8000
```

```
spring.jpa.defer-datasource-initialization=true  
spring.jpa.show-sql=true  
spring.datasource.url=jdbc:h2:mem:testdb  
spring.h2.console.enabled=true  
spring.h2.console.path=/h2-console
```

application.properties

Fichier de configuration de Spring Boot et ses dépendances. Permet de définir / modifier de manière externalisée de nombreux paramètres. Voir

<https://docs.spring.io/spring-boot/docs/current/reference/html/common-application-properties.html>

```
package fr.dauphine.miageif.msa.MSA;
...
// Classe persistente representant un "taux de change"
@Entity
public class TauxChange {
    ...
}
```

TauxChange.java

Entité JPA définissant un taux de change associée à un JPARepository Spring Data

```
package fr.dauphine.miageif.msa.exemple;
import org.springframework.data.jpa.repository.JpaRepository;
// Creation du JPA Repository basé sur Spring Data
// Permet de "générer" toutes sortes de requêtes JPA, sans implementation
public interface TauxChangeRepository extends JpaRepository<TauxChange, Long>
{
    TauxChange findBySourceAndDest(String source, String dest);
}
```

TauxChangeRepository.java

Dév. avec Spring Boot

Spring Data

projet Spring pour simplifier l'accès aux données et avoir une couche d'abstraction commune à de multiples sources de données (JDBC, JPA, MongoDB, Cassandra, Elasticsearch...) :

<http://projects.spring.io/spring-data/>

Spring Boot starter : `spring-boot-starter-data-jpa`

Pour JPA : <https://projects.spring.io/spring-data-jpa/>

Hériter de l'interface `org.springframework.data.jpa.JpaRepository`

Génération des opérations sur les données ([CRUD](#), [findxxx](#), [saveAll...](#)), sans écrire de requête, ni DAO !

```
public interface TauxChangeRepository extends JpaRepository<TauxChange, Long>
```

Premier paramètre l'entité concernée, deuxième paramètre le type d'@id.

```
TauxChange findBySourceAndDest(String source, String dest);
```

Query Method

Dév. avec Spring Boot

Base de données H2

H2 est un moteur SQL open-source écrit en Java <http://www.h2database.com>

Très léger (1 Mo)

Base de données SQL en mémoire (données perdues avec l'arrêt de l'application)

Directement intégrable dans une application Java

Très simple à mettre en place et complètement auto-configuré par Spring Boot

Utilisation ici à des fins de tests (lors du développement ou tests unitaires...)

Défini dans le
pom.xml

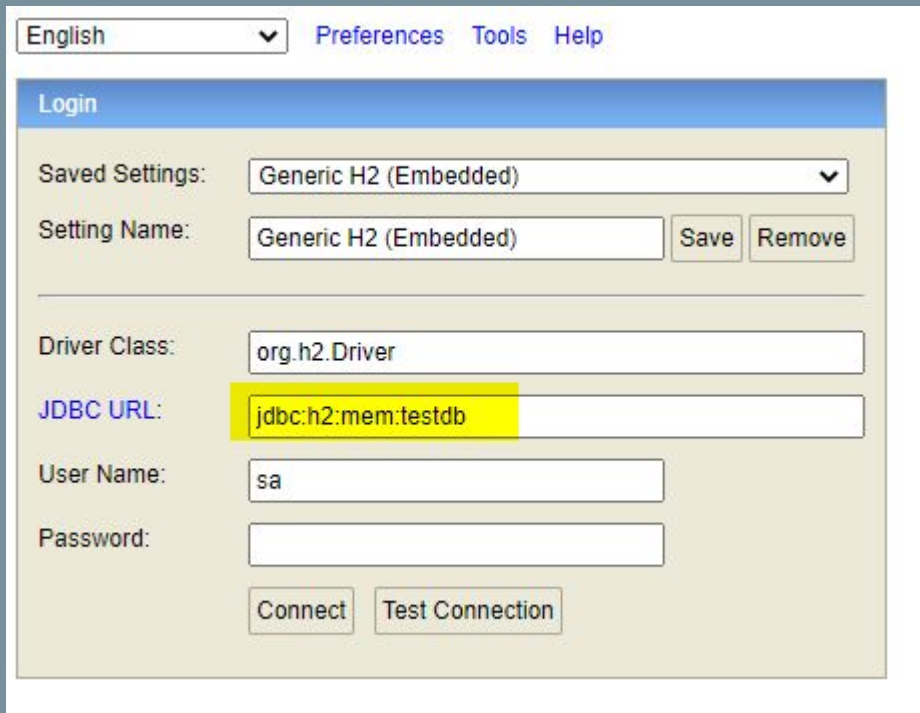
```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```


Dév. avec Spring Boot

Console de H2 à l'adresse :

<http://localhost:8000/h2-console/>

Dans le champ JDBC URL, saisir
jdbc:h2:mem:testdb pour accéder à la base



The screenshot shows the H2 Console Login interface. At the top, there is a language dropdown set to 'English' and navigation links for 'Preferences', 'Tools', and 'Help'. The main section is titled 'Login' and contains several fields and buttons. The 'Saved Settings' dropdown is set to 'Generic H2 (Embedded)'. Below it, the 'Setting Name' field also contains 'Generic H2 (Embedded)', with 'Save' and 'Remove' buttons to its right. The 'Driver Class' field is set to 'org.h2.Driver'. The 'JDBC URL' field is highlighted in yellow and contains 'jdbc:h2:mem:testdb'. The 'User Name' field is set to 'sa'. The 'Password' field is empty. At the bottom, there are 'Connect' and 'Test Connection' buttons.

English	▼	Preferences	Tools	Help
Login				
Saved Settings:	Generic H2 (Embedded) ▼			
Setting Name:	Generic H2 (Embedded)	Save	Remove	
Driver Class:	org.h2.Driver			
JDBC URL:	jdbc:h2:mem:testdb			
User Name:	sa			
Password:				
Connect		Test Connection		

Dév. avec Spring Boot

Base de données H2

Insertion de données au lancement de l'application

Définition d'un fichier **data.sql** dans le répertoire ressources de l'application

Le script sera exécuté à chaque fois que l'application est lancée

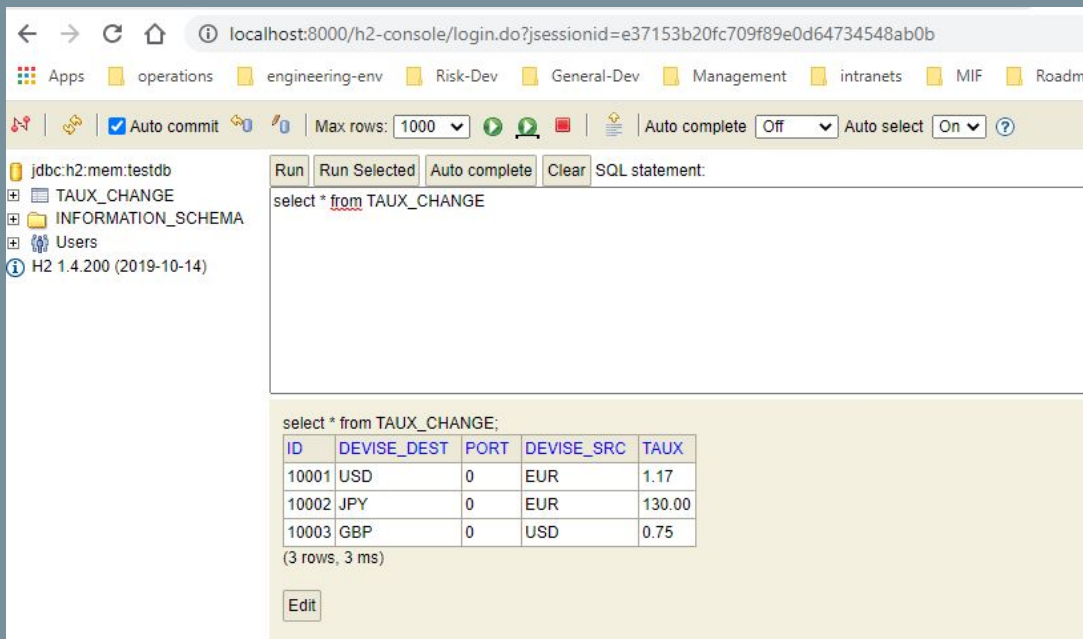
```
insert into taux_change (id,devise_src,devise_dest,taux,port)
values(10001,'EUR','USD',1.17,0);
insert into taux_change(id,devise_src,devise_dest,taux,port)
values(10002,'EUR','JPY',130,0);
insert into taux_change(id,devise_src,devise_dest,taux,port)
values(10003,'USD','GBP',0.75,0);
```

data.sql

Dév. avec Spring Boot

Console de H2 à l'adresse :

<http://localhost:8000/h2-console/>



The screenshot shows the H2 database console interface. The browser address bar displays the URL: `localhost:8000/h2-console/login.do?sessionId=e37153b20fc709f89e0d64734548ab0b`. The interface includes a navigation bar with tabs for various environments (Apps, operations, engineering-env, Risk-Dev, General-Dev, Management, intranets, MIF, Roadmap) and a toolbar with options like Auto commit, Max rows (1000), Auto complete (Off), and Auto select (On).

The left sidebar shows the database structure for `jdbc:h2:mem:testdb`, including tables `TAUX_CHANGE` and `INFORMATION_SCHEMA`, and users.

The main area displays the SQL statement: `select * from TAUX_CHANGE`. Below the statement, the results are shown in a table format:

ID	DEWISE_DEST	PORT	DEWISE_SRC	TAUX
10001	USD	0	EUR	1.17
10002	JPY	0	EUR	130.00
10003	GBP	0	USD	0.75

Below the table, it indicates `(3 rows, 3 ms)`. An `Edit` button is visible at the bottom.

Dév. avec Spring Boot

Définition d'un premier microservice REST

...

@RestController

```
public class ChangeController {
```

```
    @Autowired
```

```
    private Environment environment;
```

```
    @Autowired
```

```
    private TauxChangeRepository repository;
```

```
    @GetMapping("/devise-change/source/{source}/dest/{dest}")
```

```
    public TauxChange retrouveTauxChange
```

```
        (@PathVariable String source, @PathVariable String dest){
```

```
        TauxChange tauxChange = repository.findBySourceAndDest(source, dest);
```

```
        return tauxChange;
```

```
    }
```

```
}
```

ChangeController.java

Dév. avec Spring Boot

Définition d'un premier microservice REST

@RestController indique la classe va pouvoir traiter les requêtes que nous allons définir. Il indique aussi que chaque méthode va renvoyer directement la réponse à l'utilisateur

Ici une méthode de type GET via l'annotation `@GetMapping` sur la méthode `retrouveTauxChange`

```
GetMapping("/devise-change/source/{source}/dest/{dest}")  
public TauxChange retrouveTauxChange  
    (@PathVariable String source, @PathVariable String dest){  
    TauxChange tauxChange = repository.findBySourceAndDest(source, dest);  
    return tauxChange;  
}  
}
```

Dév. avec Spring Boot

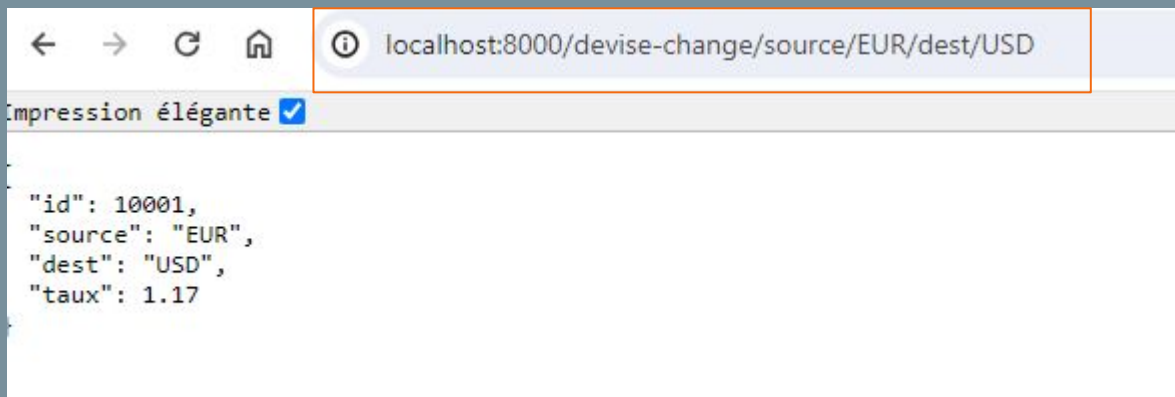
Définition d'un premier microservice REST

@RestController indique la classe va pouvoir traiter les requêtes que nous allons définir. Il indique aussi que chaque méthode va renvoyer directement la réponse à l'utilisateur

Ici une méthode de type GET via l'annotation `@GetMapping` sur la méthode `retrouveTauxChange`

```
GetMapping("/devise-change/source/{source}/dest/{dest}")  
public TauxChange retrouveTauxChange  
    (@PathVariable String source, @PathVariable String dest){  
    TauxChange tauxChange = repository.findBySourceAndDest(source, dest);  
    return tauxChange;  
}  
}
```

Dév. avec Spring Boot



<http://localhost:8000/devise-change/source/EUR/dest/USD>